UCAM

CALIDAD DE SOFTWARE

Dr. Richard Avilés López

UCAM

Tema 1

Concepto de calidad del software

Calidad del Software

Profesor Richard Avilés López

Grado en Ingeniería Informática



ÍNDICE

CONTENIDOS

1.1 Definición de Calidad del software
1.2 Las 8 dimensiones de Calidad según Garvin
1.3 Los 11 factores de Calidad según McCall
1.4 El Modelo de Boehm: hacia una visión jerárquica
1.5 La Norma ISO/IEC 9126
1.6 La Norma ISO/IEC 25010:2023
1.7 El dilema de la Calidad del Software
1.8 Economía de la Calidad del Software: Gestión y Costos en el Ciclo de Vida
1.9 La Orquestación de la Calidad

UCAM

1.1 Definición de Calidad de Software

Satisfacción de requisitos y expectativas del cliente.

1.1 Definición de Calidad de Software

Es el grado en que un sistema, componente o proceso **satisface los requisitos especificados** y **las expectativas razonables de sus interesados** (usuarios, clientes, operadores, reguladores), considerando de forma simultánea **múltiples atributos** — funcionales y no funcionales— y **no una única métrica aislada**.

La calidad de software es un **constructo multidimensional**.
Incluye un conjunto de propiedades relacionadas entre sí, y algunas pueden entrar en conflicto directo (trade-off).

$$Q = f(functionality, performance, reliability, \\ security, maintainability, flexibility, usability, safety, \dots)$$

Dos sistemas pueden "funcionar" igual de bien y, sin embargo, tener niveles de calidad radicalmente distintos.

1.1 Definición de Calidad de Software



1.1 Definición de Calidad de Software

Modelo	Época	Enfoque principal	Dimensiones
Garvin	1987	¿Desde qué **dimensiones puede percibirse la calidad** de un producto?	**8 dimensiones**: Performance, Features, Reliability, Conformance, Durability, Serviceability, Aesthetics, Perceived Quality.
McCall	1977	¿Qué **factores de calidad afectan a la operación, revisión y transición** del software?	**11 dimensiones**: Correctness, Reliability, Efficiency, Integrity, Usability, Maintainability, Flexibility, Testability, Portability, Reusability, Interoperability.
Boehm	1978	¿Cómo se **relacionan jerárquica y causalmente** las propiedades de calidad entre sí?	***Relación jerárquica***: Utility, Maintainability, Portability, Reliability, Efficiency, Usability, Testability, Understandability, Modifiability.
ISO/IEC 9126	2001-2004	¿Cómo se puede **normalizar internacionalmente** la descripción de la calidad del software?	Functionality, Reliability, Usability, Efficiency, Maintainability y Portability.
ISO/IEC 25010 Estándar actual.	2011, revisada en 2023	¿Cuál es el **marco de referencia actual para especificar, estructurar, medir y evaluar** la calidad de productos ICT y software?	Familia de estándares **SQuaRE** (***Software product Quality Requirements and Evaluation***)

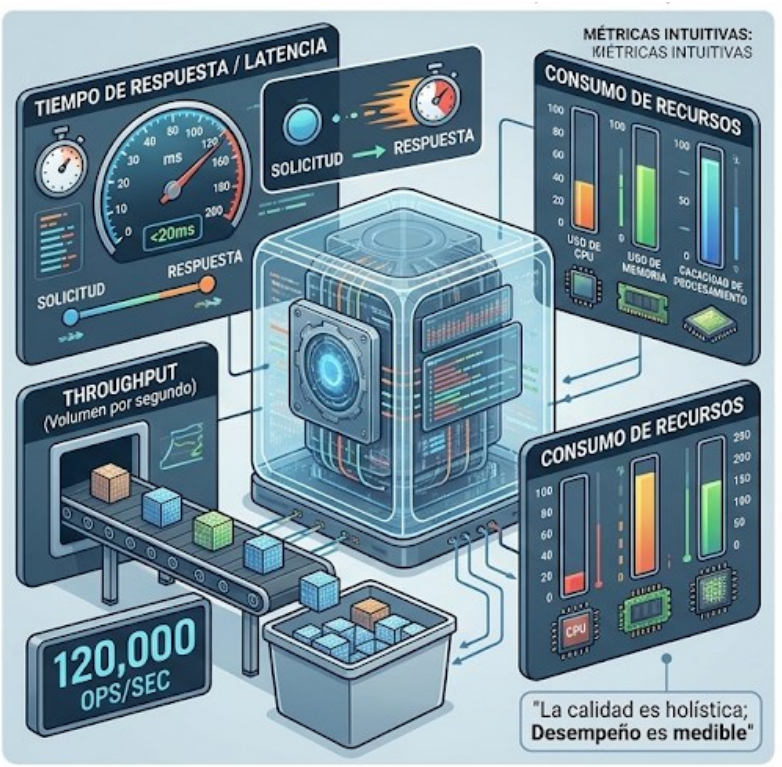
UCAM

1.2 Las 8 dimensiones de Calidad según Garvin

La calidad es multidimensional, y algunas de sus dimensiones pueden entrar en conflicto directo entre sí.

1.2 Las 8 dimensiones de Calidad según Garvin

No fueron diseñadas pensando en sistemas informáticos, resultan extraordinariamente útiles para introducir la idea de que la calidad ***no es un concepto único, sino un conjunto de perspectivas complementarias*** —y **a veces contradictorias**— sobre un mismo producto. Se adaptaron al software entre 2001-2005 por Roger Pressman.



Performance (Desempeño)

El desempeño representa el **comportamiento operativo principal del producto**: qué tan bien realiza la función para la que fue diseñado, medido en términos cuantificables.

En software, esta idea se traduce en magnitudes como:

- ✓ tiempo de respuesta, latencia
- ✓ throughput (volumen real de operaciones procesadas por unidad de tiempo),
- ✓ capacidad de procesamiento (límite teórico potencial)
- ✓ precisión de los resultados
- ✓ consumo de recursos

1.2 Las 8 dimensiones de Calidad según Garvin

No confundir desempeño (performance) con funcionalidad.

Solicitud: consultar saldo
Sistema A → responde en 150 ms
Sistema B → responde en 8 s

Ambos sistemas pueden ser funcionalmente correctos (el saldo devuelto es exacto en los dos casos). Sin embargo, desde el punto de vista del desempeño, claramente no son equivalentes.

Principio. Un atributo de calidad solo resulta útil desde el punto de vista de la ingeniería cuando puede transformarse, siempre que sea razonable, ***en una condición verificable*** —es decir, en un ***requisito con un umbral, una métrica y un método de medición asociados***.

"*La aplicación es rápida*" no constituye una especificación de ingeniería. Es una apreciación subjetiva, no verificable. Debe transformarse en una condición medible.



1.2 Las 8 dimensiones de Calidad según Garvin

Features (Características)

Las características ***son funcionalidades adicionales que complementan las capacidades básicas o esenciales del producto***, ampliando lo que el usuario puede hacer con él.

En una aplicación bancaria, por ejemplo, la ***funcionalidad básica*** podría ser "*consultar saldo y realizar transferencias*", mientras que serían ***características*** adicionales:

- ✓ categorización automática de gastos;
- ✓ notificaciones push;
- ✓ análisis financiero personalizado;
- ✓ integración con cuentas de otras entidades.



1.2 Las 8 dimensiones de Calidad según Garvin

Features (Características)

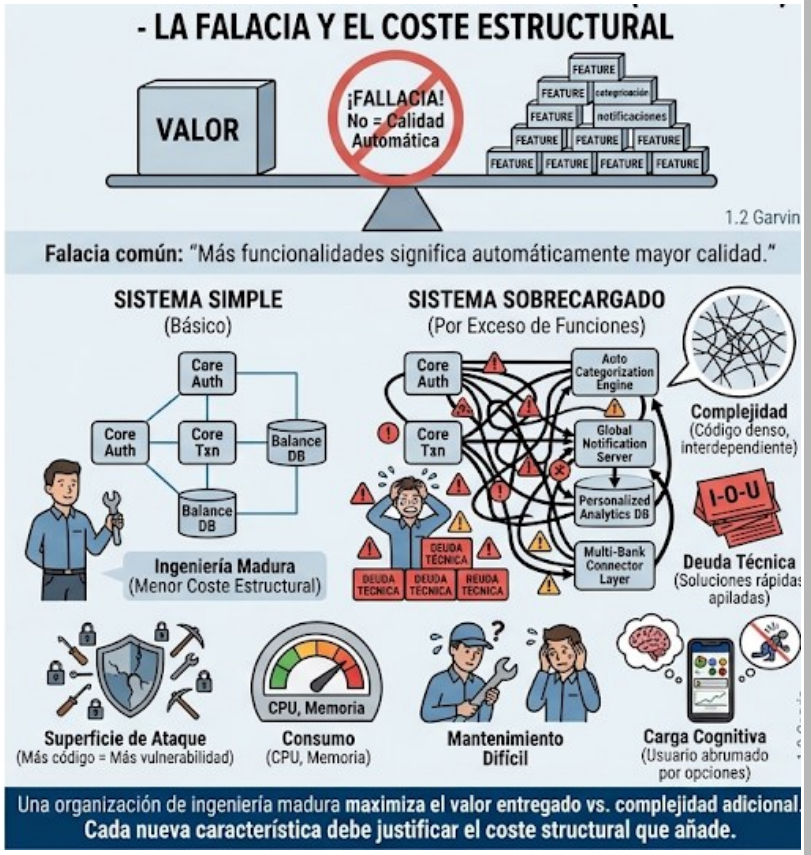
Falacia común: "*Más funcionalidades significa automáticamente mayor calidad.*"

Añadir funcionalidades tiende a incrementar simultáneamente:

- ✓ la complejidad del sistema;
- ✓ la deuda técnica acumulada;
- ✓ la superficie de ataque (más código expuesto, más vectores de vulnerabilidad);
- ✓ el consumo de recursos;
- ✓ la dificultad de mantenimiento;
- ✓ la carga cognitiva que el usuario debe soportar para operar el sistema.

Una organización de ingeniería madura **no intenta maximizar el número funcionalidades**, sino **el valor entregado** en relación con **la complejidad adicional que cada funcionalidad incorpora**.

Por ello, **cada nueva característica debe justificar el coste estructural que añade al sistema**.



1.2 Las 8 dimensiones de Calidad según Garvin

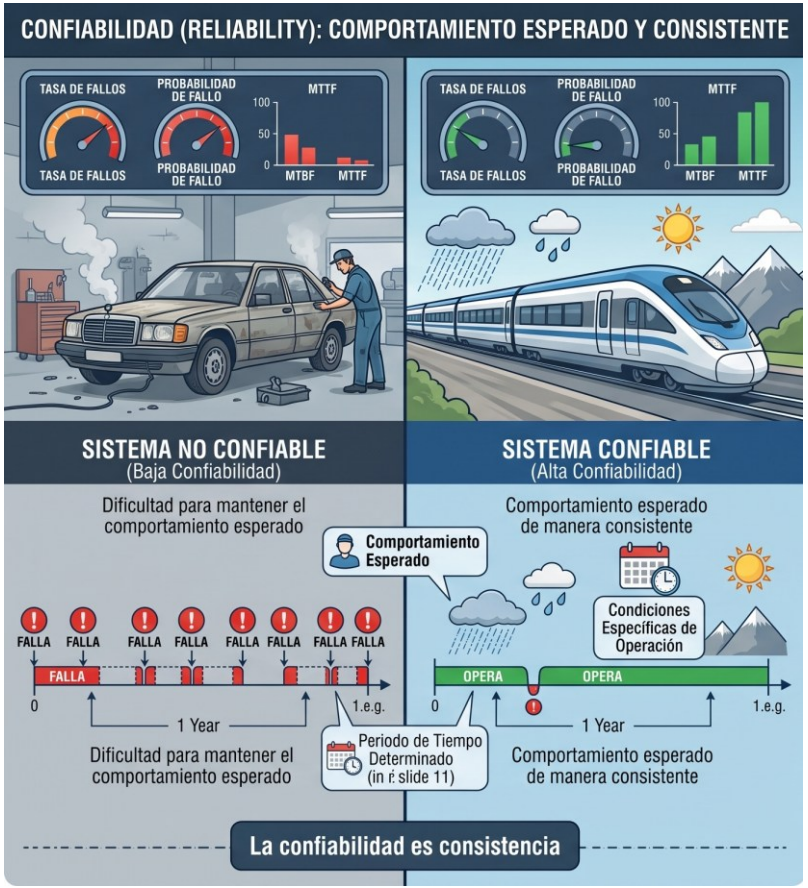
Reliability (Confiabilidad)

La confiabilidad es ***la capacidad de un sistema para proporcionar el comportamiento esperado de manera consistente durante un periodo de tiempo determinado y bajo condiciones específicas de operación***.

Se analiza habitualmente mediante conceptos como:

- ✓ probabilidad de fallo
- ✓ tasa de fallos
- ✓ disponibilidad (availability)
- ✓ MTBF (Mean Time Between Failures, tiempo medio entre fallos) para partes reparables.
- ✓ MTTF (Mean Time To Failure, tiempo medio hasta el fallo) para partes no reparables.

Es crucial distinguir con precisión dos términos que suelen confundirse: ***Confiabilidad ≠ Disponibilidad***.



1.2 Las 8 dimensiones de Calidad según Garvin

Reliability (Confiabilidad)

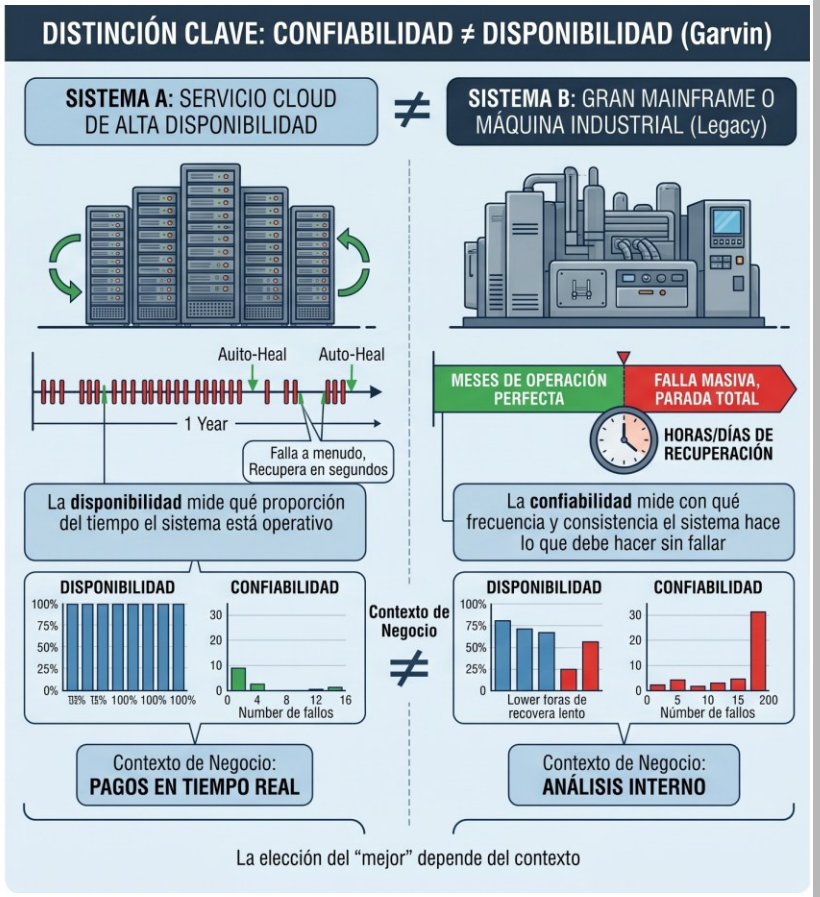
La ***confiabilidad*** mide ***con qué frecuencia y consistencia el sistema hace lo que debe hacer sin fallar***.

La ***disponibilidad*** mide qué ***proporción del tiempo total el sistema está operativo y accesible***, independientemente de cuántas veces haya fallado, siempre que se recupere rápido.

Un ***sistema A*** podría presentar una disponibilidad muy elevada precisamente porque se recupera con extraordinaria rapidez cada vez que falla (falla a menudo, pero se repara en segundos).

Un ***sistema B*** podría fallar muy raramente, pero necesitar horas para recuperarse cuando lo hace.

Ambos sistemas presentan **perfiles de calidad completamente distintos, y la elección de cuál es "mejor" depende del contexto de negocio**: un sistema de pagos en tiempo real puede tolerar peor la larga recuperación del sistema B, mientras que un sistema de análisis interno podría tolerarla sin problema.



1.2 Las 8 dimensiones de Calidad según Garvin

Conformance (Conformidad)

La conformidad es el grado en que el producto cumple con sus especificaciones, contratos, estándares, requisitos, regulaciones y políticas organizativas aplicables.

La conformidad suele ser la suma de varias fuentes de exigencia:

$$Requisitos\ funcionales\ +\ Requisitos\ de\ calidad\ +\ Requisitos\ legales\ \ +\ Requisitos\ de\ seguridad\ +\ Políticas\ corporativas\ =\ Conformidad$$

Un sistema técnicamente impresionante que **incumpla una obligación regulatoria no puede considerarse un éxito desde el punto de vista organizativo**, por brillante que sea su arquitectura o su rendimiento.

La conformidad actúa, en la práctica, **como una condición de aceptación previa a cualquier otra evaluación de calidad**.

1.2 Las 8 dimensiones de Calidad según Garvin



1.2 Las 8 dimensiones de Calidad según Garvin

Durability (Durabilidad)

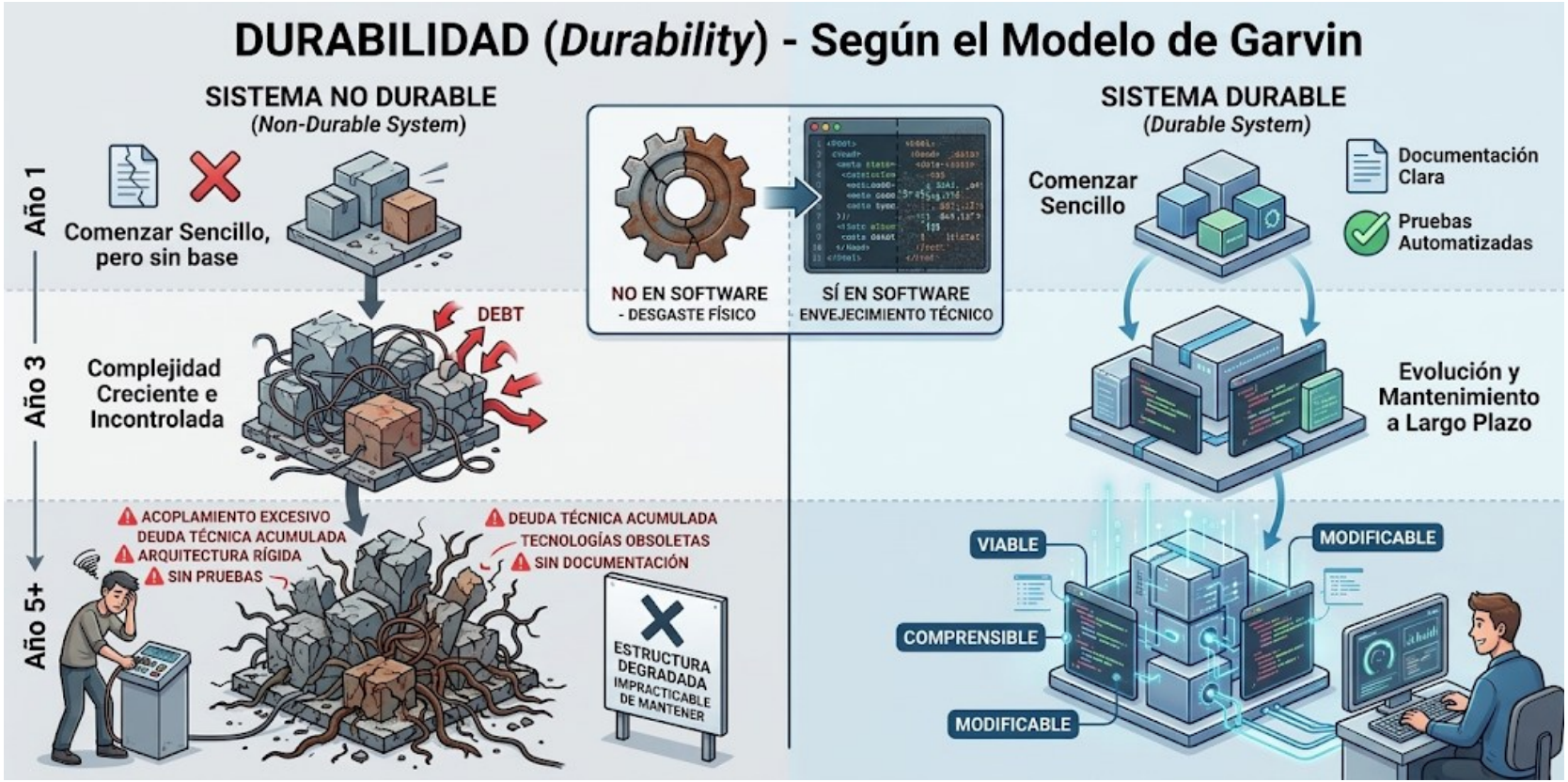
La durabilidad es la capacidad del software para **seguir siendo viable, comprensible y modificable a lo largo del tiempo, sin degradarse estructuralmente** hasta el punto de volverse impracticable de mantener.

El software no se desgasta físicamente como una pieza mecánica, pero sí puede envejecer técnicamente.

Por tanto, la **durabilidad** debe entenderse como una **propiedad estrechamente ligada a la capacidad de evolución y mantenimiento a largo plazo**, y no como resistencia física al uso.

Un sistema puede comenzar siendo relativamente sencillo y ***convertirse gradualmente en un producto extremadamente difícil de modificar***, debido a: acoplamiento excesivo entre componentes; dependencia de tecnologías obsoletas; deuda técnica acumulada; arquitectura rígida, no preparada para el cambio; ausencia de pruebas automatizadas; falta de documentación; y complejidad creciente y no controlada.

1.2 Las 8 dimensiones de Calidad según Garvin



1.2 Las 8 dimensiones de Calidad según Garvin

Serviceability (Capacidad de Servicio)

La serviceability es la facilidad con la que un sistema puede ser ***diagnosticado, corregido y restaurado a un estado operativo correcto*** tras la aparición de un problema. Esta dimensión conecta directamente con las **prácticas modernas de observabilidad** (logging, métricas, trazabilidad) y de **operación de sistemas** (SRE, DevOps).

Ejemplo: Un servicio recibe una incidencia crítica a las 03:00 de la madrugada. Dos sistemas presentan exactamente el mismo defecto técnico.

En el ***sistema A*** existen: registros estructurados (logs), métricas, trazas distribuidas, alertas automáticas, paneles de control (dashboards) y mecanismos de reversión (rollback).

En el ***sistema B*** solamente existe un fichero de texto plano con mensajes ambiguos y sin contexto.

El **defecto es idéntico en ambos sistemas**. La **capacidad de servicio no lo es en absoluto**: el equipo que atiende el sistema A puede diagnosticar y resolver el problema en minutos; el equipo que atiende el sistema B puede tardar horas.

1.2 Las 8 dimensiones de Calidad según Garvin



1.2 Las 8 dimensiones de Calidad según Garvin

Aesthetics (Estética)

La estética es la **percepción sensorial y formal del producto**, referida a su **coherencia visual, diseño, consistencia y claridad de presentación**.

En software moderno, la estética está estrechamente relacionada con la experiencia de usuario (UX), el diseño visual y la percepción general de calidad, pero es importante evitar otro error habitual:

Falacia común: "Una interfaz atractiva compensa problemas técnicos serios."

Una interfaz visualmente cuidada no compensa problemas graves de seguridad, confiabilidad o rendimiento; en el mejor de los casos, puede retrasar el momento en que el usuario los percibe.

La verdadera calidad requiere tanto belleza visual como integridad estructural.

1.2 Las 8 dimensiones de Calidad según Garvin



La verdadera **calidad** requiere tanto **belleza visual** como **integridad estructural**.

1.2 Las 8 dimensiones de Calidad según Garvin

Perceived Quality (Calidad Percibida)

La calidad percibida es la **evaluación subjetiva** que el usuario realiza **sobre el producto**, **basada en la información** —completa o incompleta— de la que **dispone**, y **no necesariamente en un análisis técnico riguroso**.

La calidad técnica y la calidad percibida no son necesariamente idénticas.

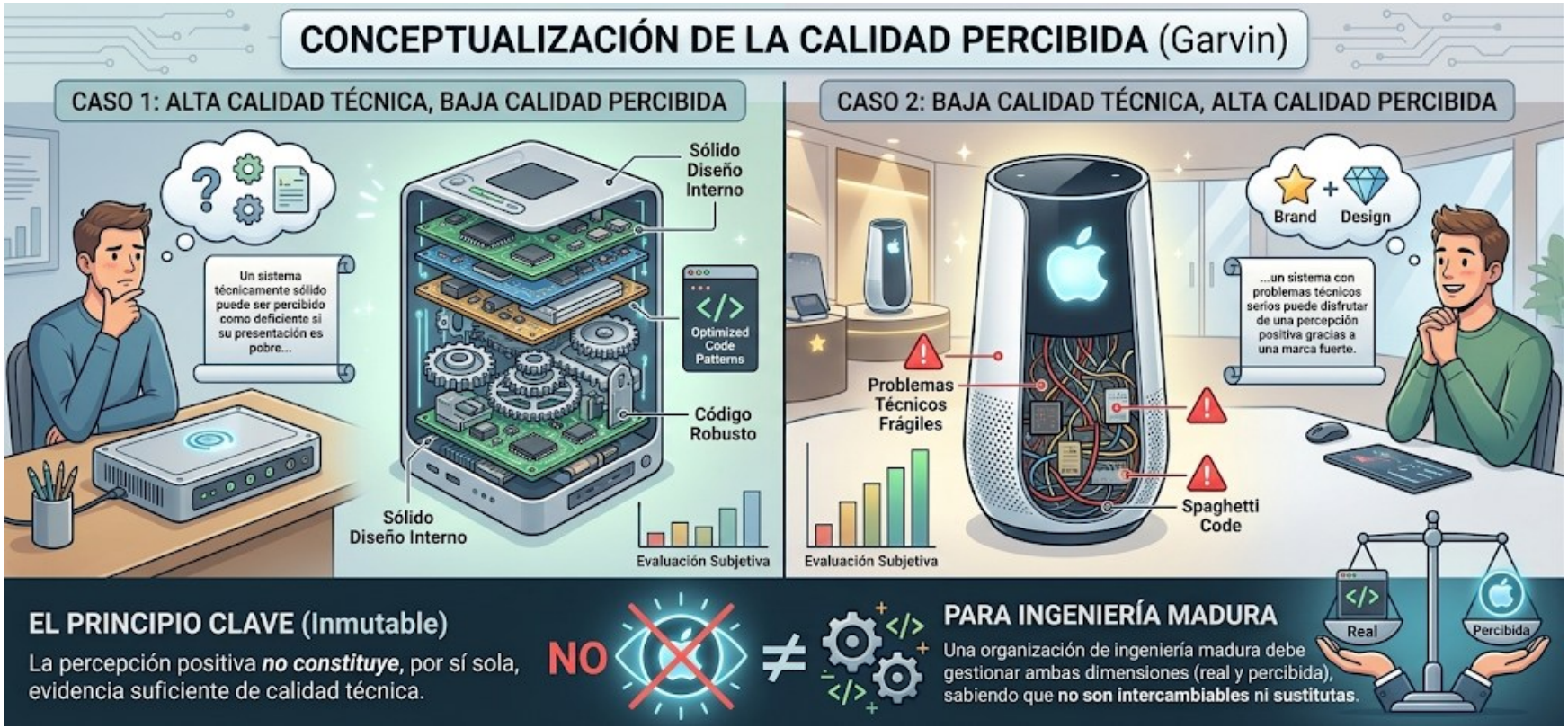
Un sistema técnicamente sólido puede ser percibido como deficiente si su presentación es pobre; y, a la inversa, un sistema con problemas técnicos serios puede disfrutar de una percepción positiva gracias a una marca fuerte.

Principio.

La percepción positiva no constituye, por sí sola, evidencia suficiente de calidad técnica.

Una organización de ingeniería madura debe ***gestionar ambas dimensiones*** —la calidad técnica real y la calidad percibida— ***sabiendo que no son intercambiables ni sustitutas*** la una de la otra.

1.2 Las 8 dimensiones de Calidad según Garvin



1.2 Las 8 dimensiones de Calidad según Garvin

La aportación de Garvin es conceptual y profunda. Consiste en demostrar que:

La calidad es multidimensional, y algunas de sus dimensiones pueden entrar en conflicto directo entre sí.

Por ejemplo:

- ✓ más funcionalidades → más complejidad estructural → posible menor mantenibilidad.
- ✓ Máxima personalización → más opciones disponibles para el usuario → mayor capacidad configurable → Mayor carga cognitiva para quien debe decidir.

Gestionar estos compromisos es una de las competencias centrales que debe desarrollar todo ingeniero de software, y es precisamente lo que distingue una decisión de diseño informada de una decisión tomada por inercia.

UCAM

1.3 Los 11 factores de Calidad según McCall

Evalúa el SW en su entorno de ejecución y su capacidad de despliegue en diferentes estructuras. Se enfoca en su evolución.

1.3 Los 11 factores de Calidad según McCall

McCall organiza 11 factores de calidad en 3 grandes categorías, cada una asociada a una etapa distinta del ciclo de vida del sistema:

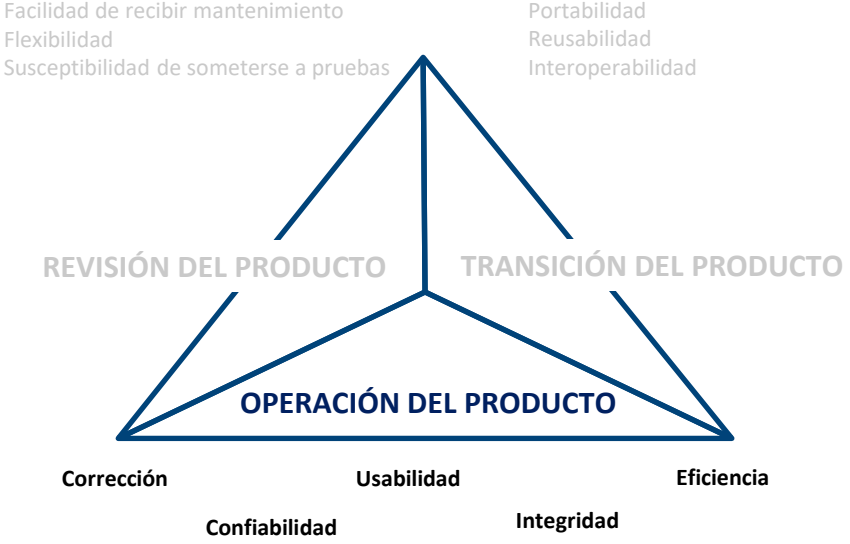
Categoría	*Pregunta que responde*	*Factores incluidos*
Operación del producto	***Evalúa el software en su entorno de ejecución***. Responde a: ¿Funciona bien el sistema mientras se usa?	***Corrección, Confiabilidad, Eficiencia, Integridad, Usabilidad***
Revisión del producto	**Se enfoca en la evolución del software**, respondiendo a la pregunta: ¿Es fácil modificarlo y verificarlo?	***Mantenibilidad, Flexibilidad, Testabilidad***
Transición del producto	***Evalúa la capacidad de despliegue en diferentes infraestructuras***, respondiendo a la pregunta: ¿Se adapta bien a nuevos entornos y contextos?	***Portabilidad, Reusabilidad, Interoperabilidad***

1.3 Los 11 factores de Calidad según McCall

Operación del producto



Diagrama de los factores de la calidad de McCall



Nota. Esta figura muestra todos los factores de calidad según McCall

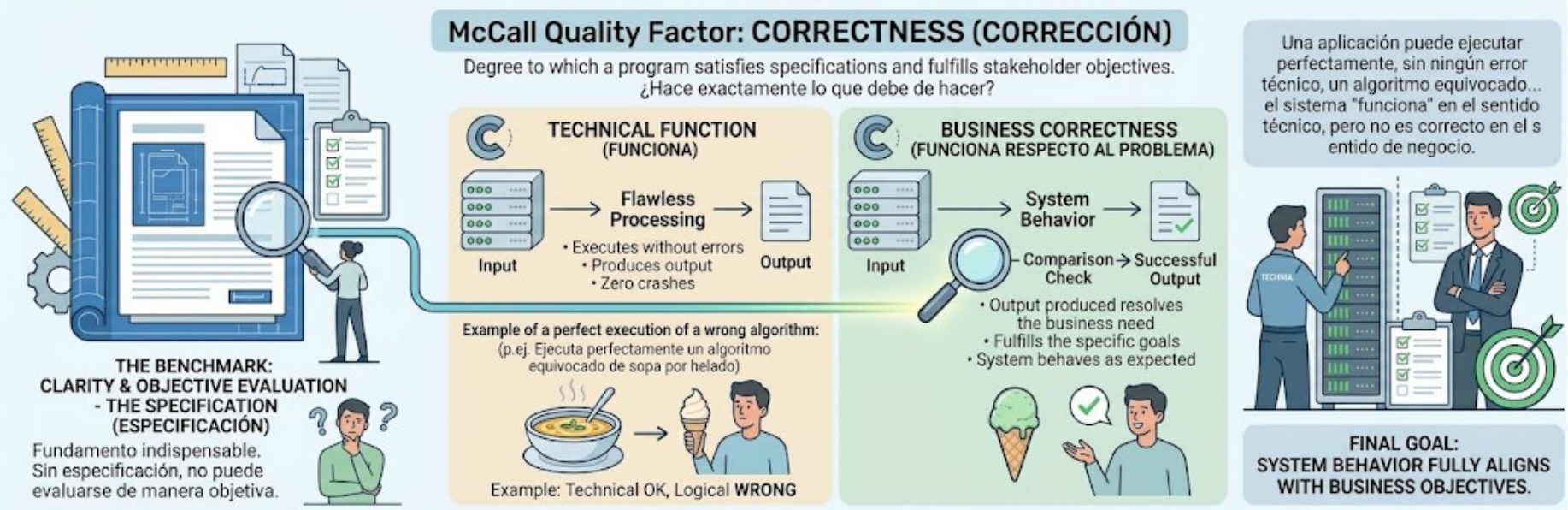
1.3 Los 11 factores de Calidad según McCall

Operación del producto

Evalúa factores de calidad en su entorno de ejecución, desde la ***perspectiva del usuario final y del entorno de producción***, midiendo la ***conformidad funcional y el rendimiento bajo condiciones operativas normales y anormales***.

- **Correctness (Corrección)**: La medida en que un programa ***satisface sus especificaciones y cumple con los objetivos*** de los stakeholders. ¿Hace exactamente lo que debe de hacer?
    - Funciona: El sistema se ejecuta sin errores, produce una salida, no se detiene inesperadamente.
    - Funciona correctamente respecto al problema que debía resolver: la salida producida es la que realmente resuelve la necesidad de negocio.
- Una aplicación puede ejecutar perfectamente, sin ningún error técnico, un algoritmo equivocado. El sistema "funciona" en el sentido técnico, pero no es correcto en el sentido de negocio. Por esta razón, la corrección depende fundamentalmente de que exista una especificación clara contra la cual contrastar el comportamiento del sistema; **sin especificación, no puede evaluarse la corrección de manera objetiva**.

1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall

Operación del producto

- **Reliability (Confiabilidad)**: La probabilidad de que el software ***ejecute su función esperada sin fallos en un entorno específico y durante un período de tiempo determinado***. Implica la tolerancia a fallos, la resiliencia y la madurez del sistema frente a estados de excepción. Ver lámina 12.

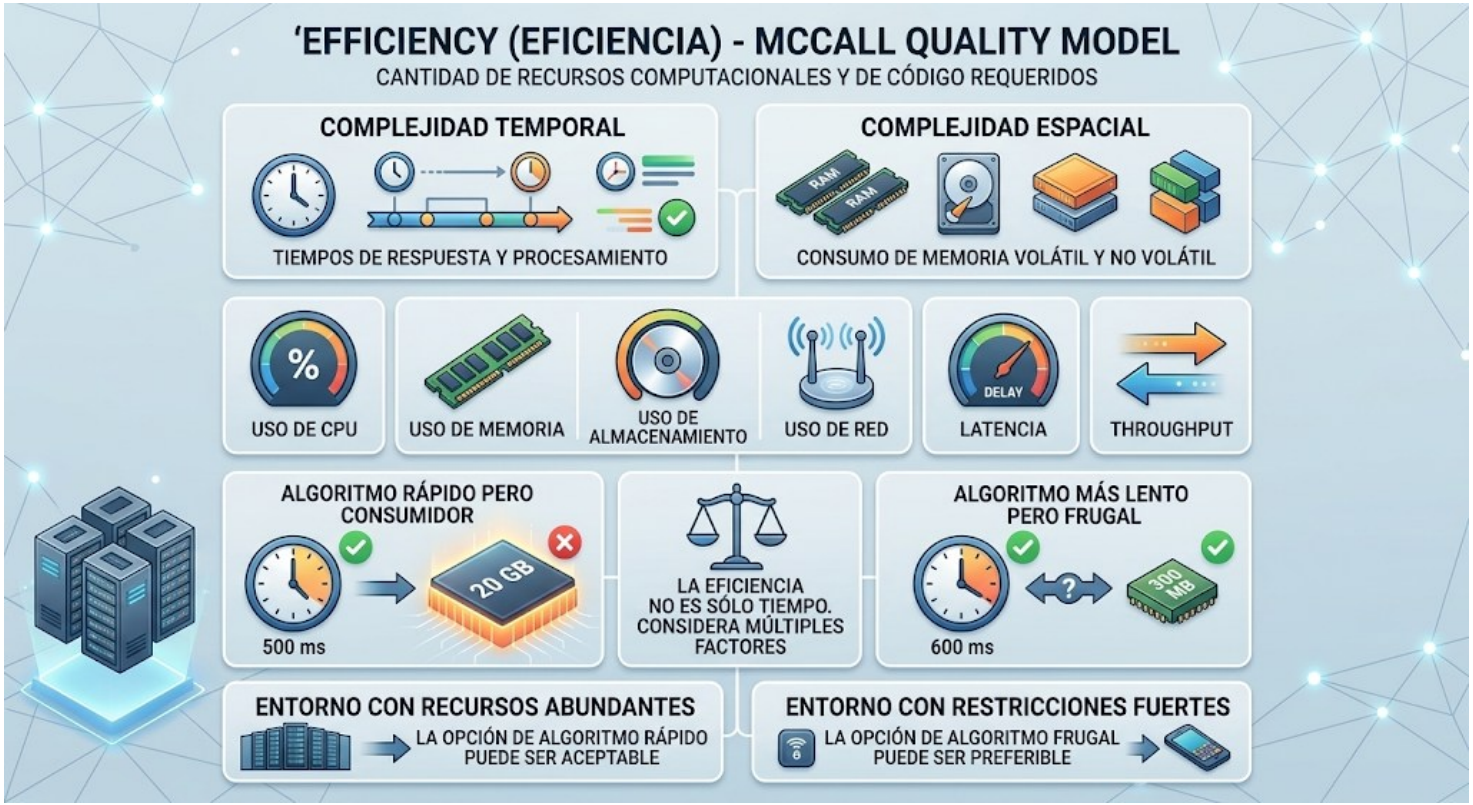
- Se cuantifica típicamente a través del ***Tiempo Medio Hasta el Fallo*** (MTTF - Mean Time To Failure). Implica la tolerancia a fallos, la resiliencia y la madurez del sistema frente a estados de excepción.
- Ejemplo:
- *Un sistema de nóminas puede funcionar durante meses sin sufrir ninguna caída y, sin embargo, calcular incorrectamente ciertos salarios en casos particulares (por ejemplo, empleados con contratos a tiempo parcial). Desde el punto de vista empresarial, esto representa un problema de* ***confiabilidad funcional****, aunque el proceso técnico de ejecución no produzca ninguna excepción visible. Este matiz es importante:* ***la confiabilidad, en este modelo, no se limita a "no caerse", sino que incluye la precisión sostenida del comportamiento****.*

1.3 Los 11 factores de Calidad según McCall

Operación del producto

- **Efficiency (Eficiencia)**: La ***cantidad de recursos computacionales y de código requeridos por un programa para realizar su función***. Conceptos asociados: Complejidad temporal (tiempos de respuesta y procesamiento) y complejidad espacial (consumo de memoria volátil y no volátil).
- Debe considerar simultáneamente magnitudes como:
  - ✓ uso de CPU; uso de memoria; uso de almacenamiento; uso de red;
  - ✓ latencia; throughput.
- No basta con preguntar "¿cuánto tarda?". También es imprescindible preguntar "¿cuántos recursos consume para conseguirlo?".
- Un algoritmo que responde en 500 ms consumiendo 20 GB de memoria no es equivalente, desde el punto de vista de la eficiencia, a otro que responde en 600 ms utilizando 300 MB.
- La calidad, una vez más, es contextual: en un entorno con recursos abundantes, la primera opción puede ser aceptable; en un entorno con restricciones fuertes de infraestructura o coste, la segunda puede ser claramente preferible.

1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall

Operación del producto

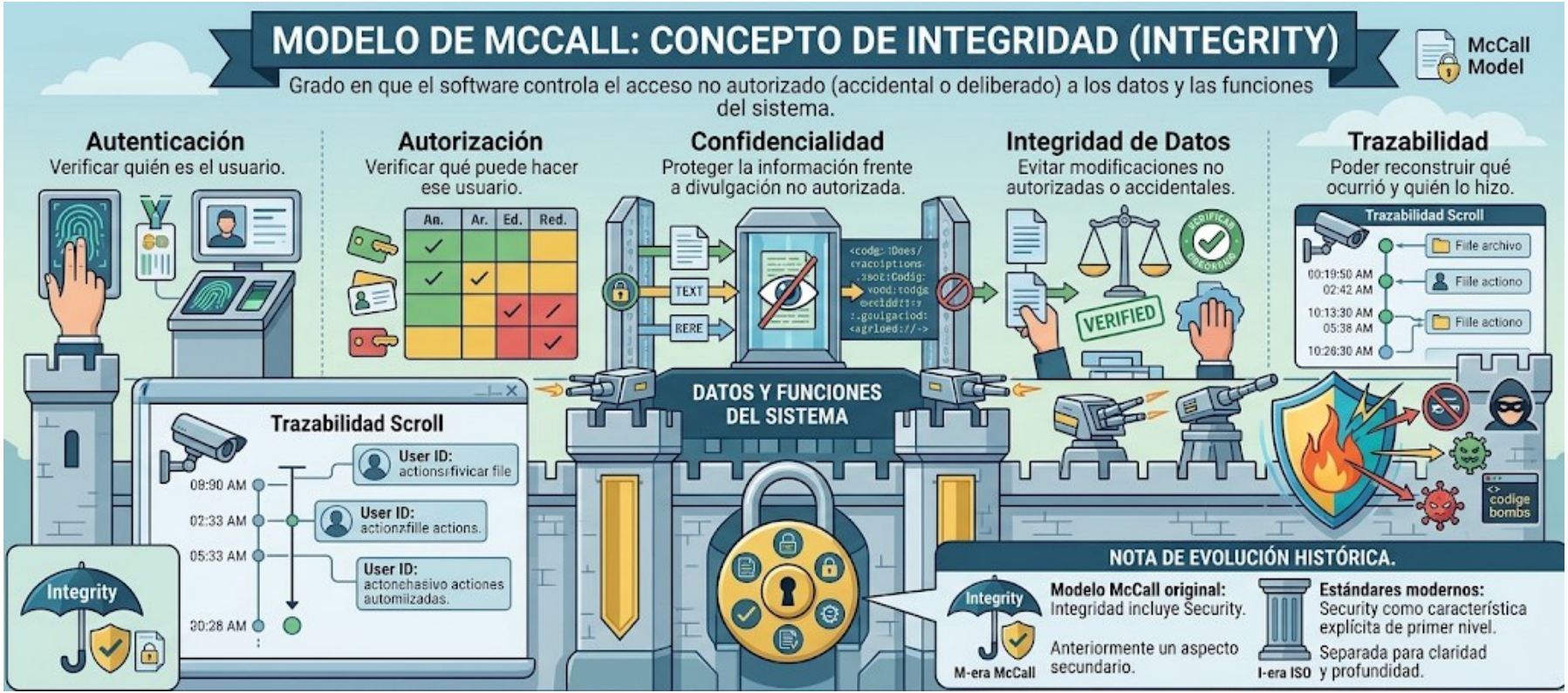
- **Integrity (Integridad)**: El grado en que el software ***controla el acceso no autorizado*** (tanto accidental como deliberado) a los datos y a las funciones del sistema.

Incluye:

- ✓ autenticación (verificar quién es el usuario);
- ✓ autorización (verificar qué puede hacer ese usuario);
- ✓ confidencialidad (proteger la información frente a divulgación no autorizada);
- ✓ integridad de datos (evitar modificaciones no autorizadas o accidentales);
- ✓ trazabilidad (poder reconstruir qué ocurrió y quién lo hizo);
- ✓ protección frente a ataques.

Esta evolución histórica ayuda a entender por qué los estándares modernos —como se verá más adelante con ISO/IEC 25010— separan Security como una característica explícita y de primer nivel, en lugar de tratarla como un aspecto secundario de la integridad.

1.3 Los 11 factores de Calidad según McCall



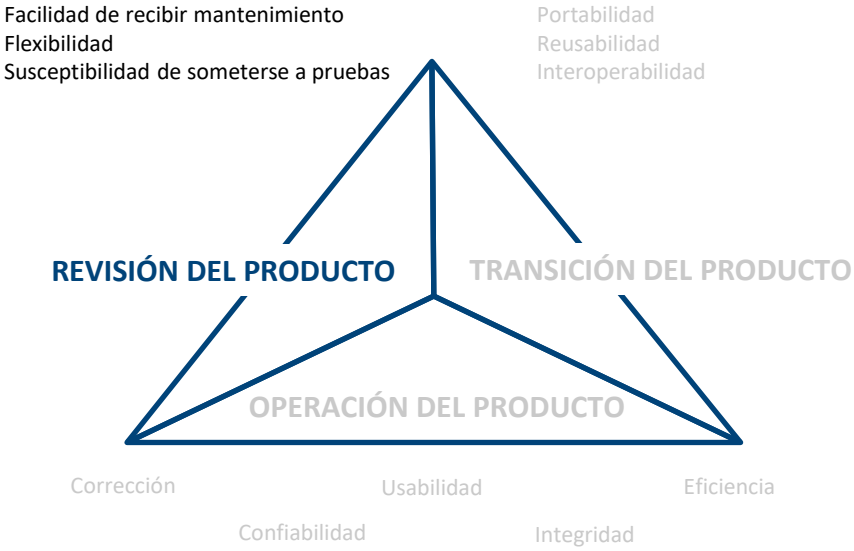
1.3 Los 11 factores de Calidad según McCall

Revisión del producto

- **Facilidad de recibir mantenimiento**: esfuerzo para detectar y corregir un error en un programa.
- **Flexibilidad**: esfuerzo necesario para modificar un programa que ya opera.
- **Susceptibilidad de someterse a pruebas:** esfuerzo para garantizar que se cumple con la funcionalidad.

Figura 14

Diagrama de los factores de la calidad de McCall



Nota. Esta figura muestra todos los factores de calidad según McCall

1.3 Los 11 factores de Calidad según McCall

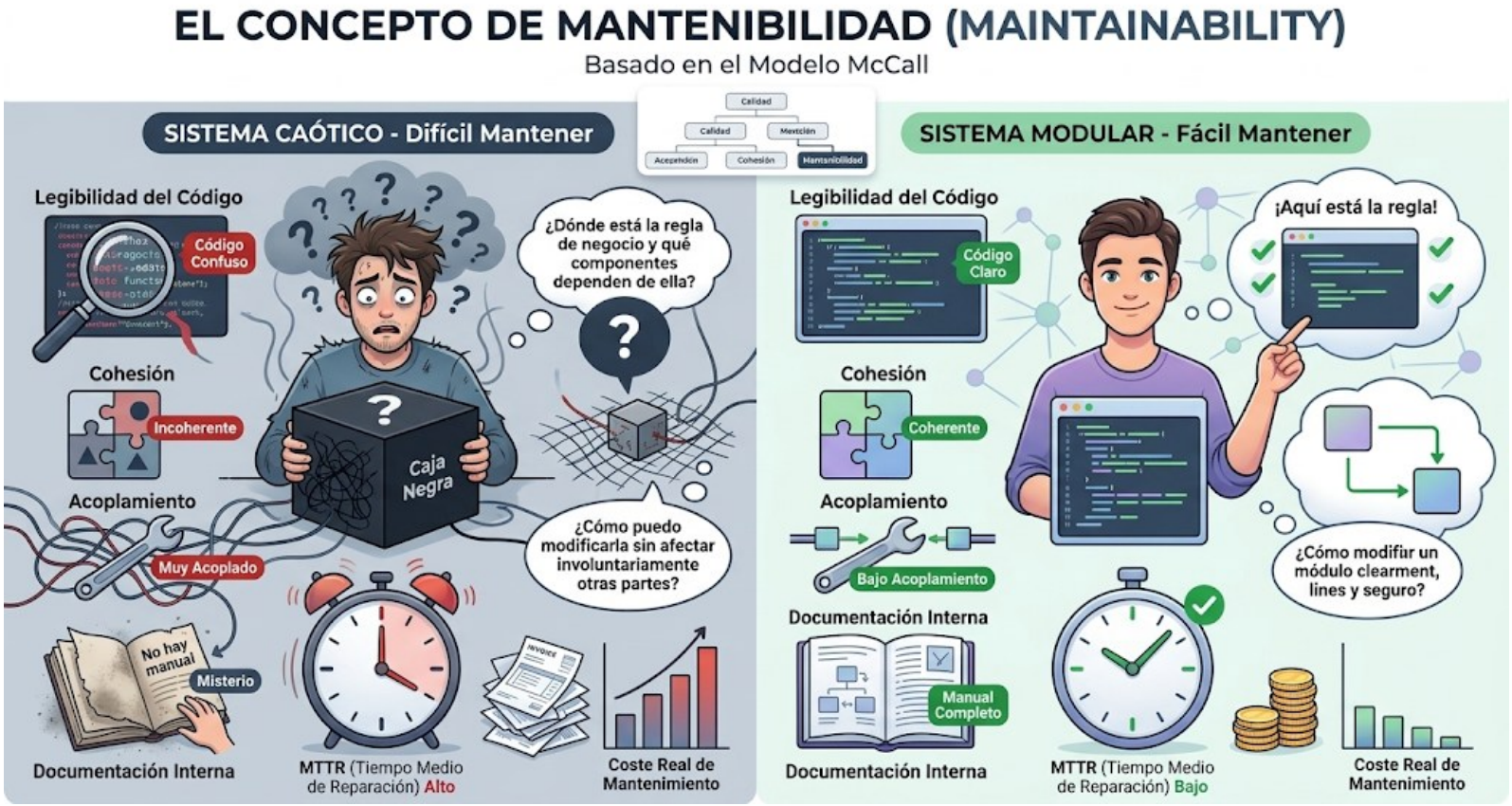
Revisión del producto

Evalúa la evolución del sistema desde la perspectiva del ingeniero de mantenimiento y del desarrollador

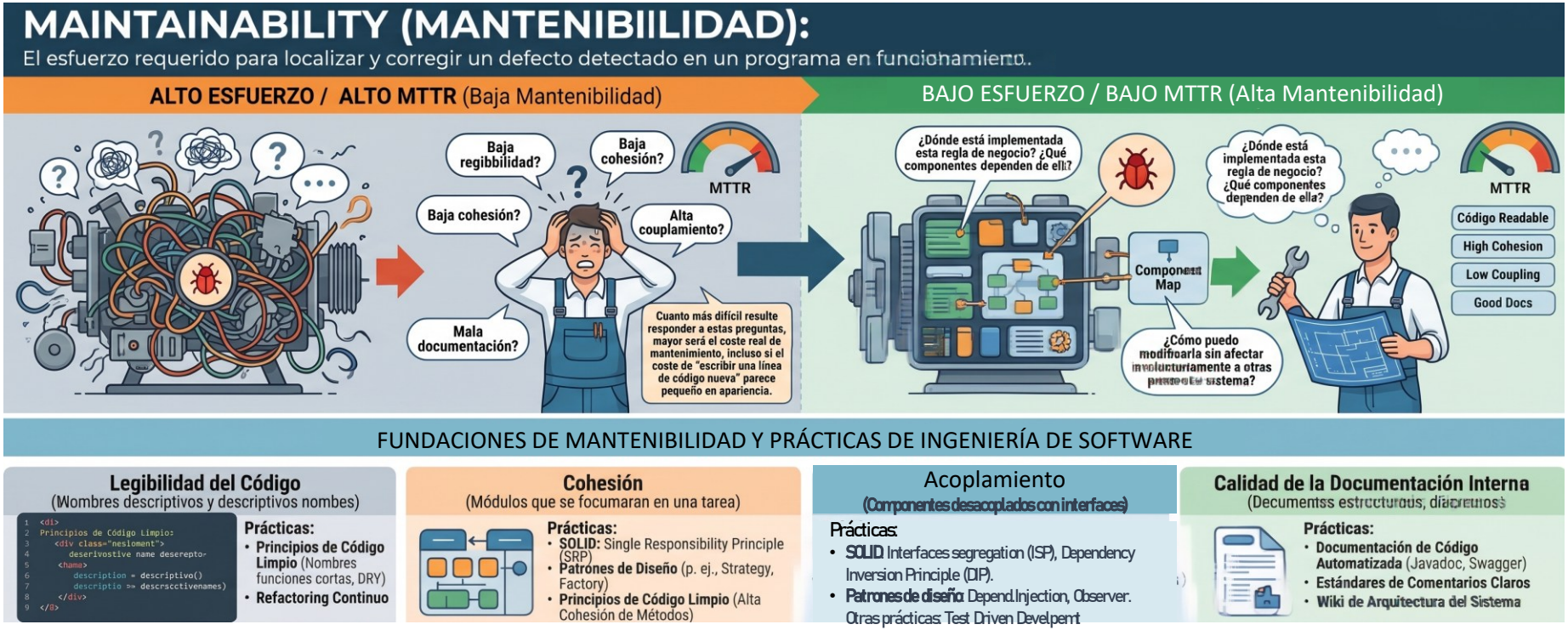
- **Maintainability (Mantenibilidad)**: El esfuerzo requerido para localizar y corregir un defecto detectado en un programa en funcionamiento.

- Se mide convencionalmente mediante el Tiempo Medio de Reparación (MTTR - Mean Time To Repair). Depende directamente de la legibilidad del código, la cohesión, el acoplamiento y la calidad de la documentación interna. Un sistema mantenible permite responder con relativa facilidad a preguntas como:
  - ✓ ¿Dónde está implementada esta regla de negocio? ¿Qué componentes dependen de ella?
  - ✓ ¿Cómo puedo modificarla sin afectar involuntariamente a otras partes del sistema?
- Cuanto más difícil resulte responder a estas preguntas, mayor será el coste real de mantenimiento, incluso si el coste de "escribir una línea de código nueva" parece pequeño en apariencia.

1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall



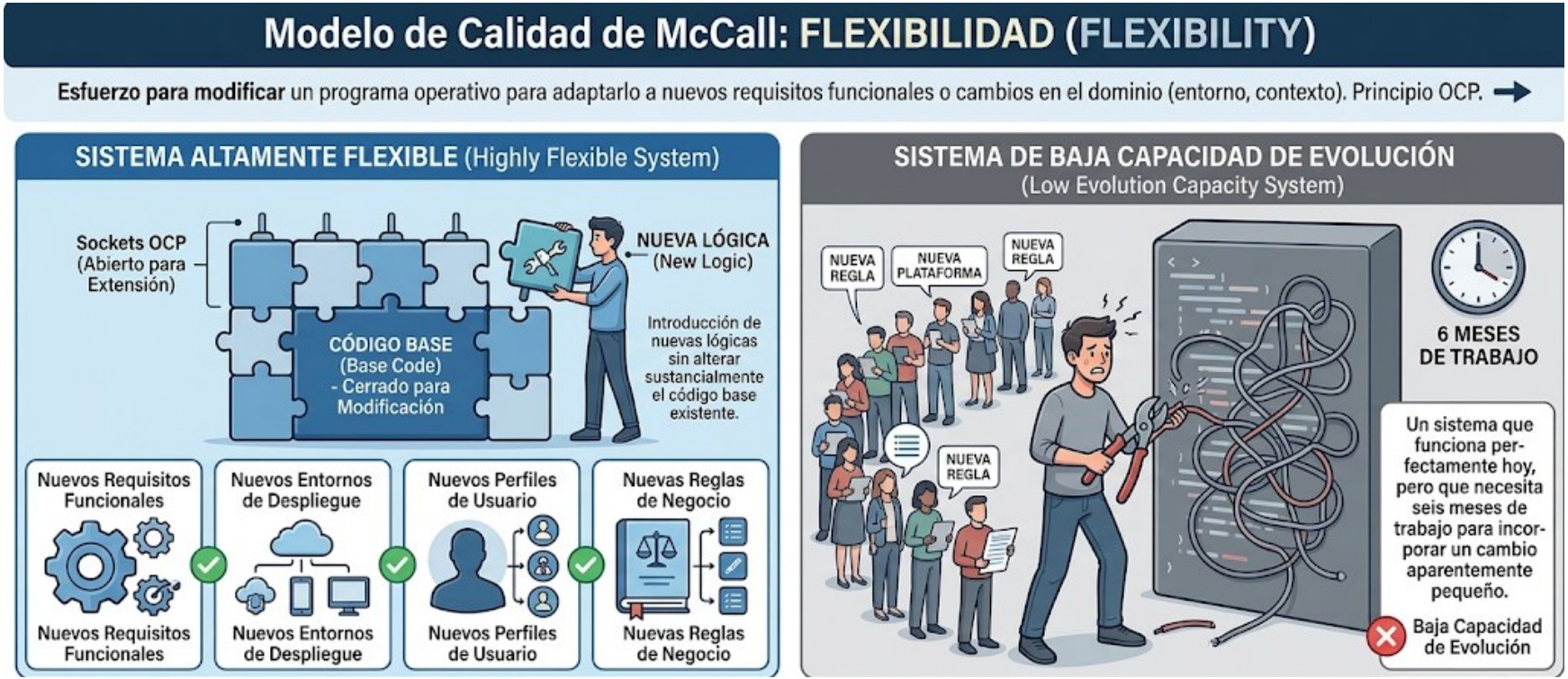
1.3 Los 11 factores de Calidad según McCall

Revisión del producto

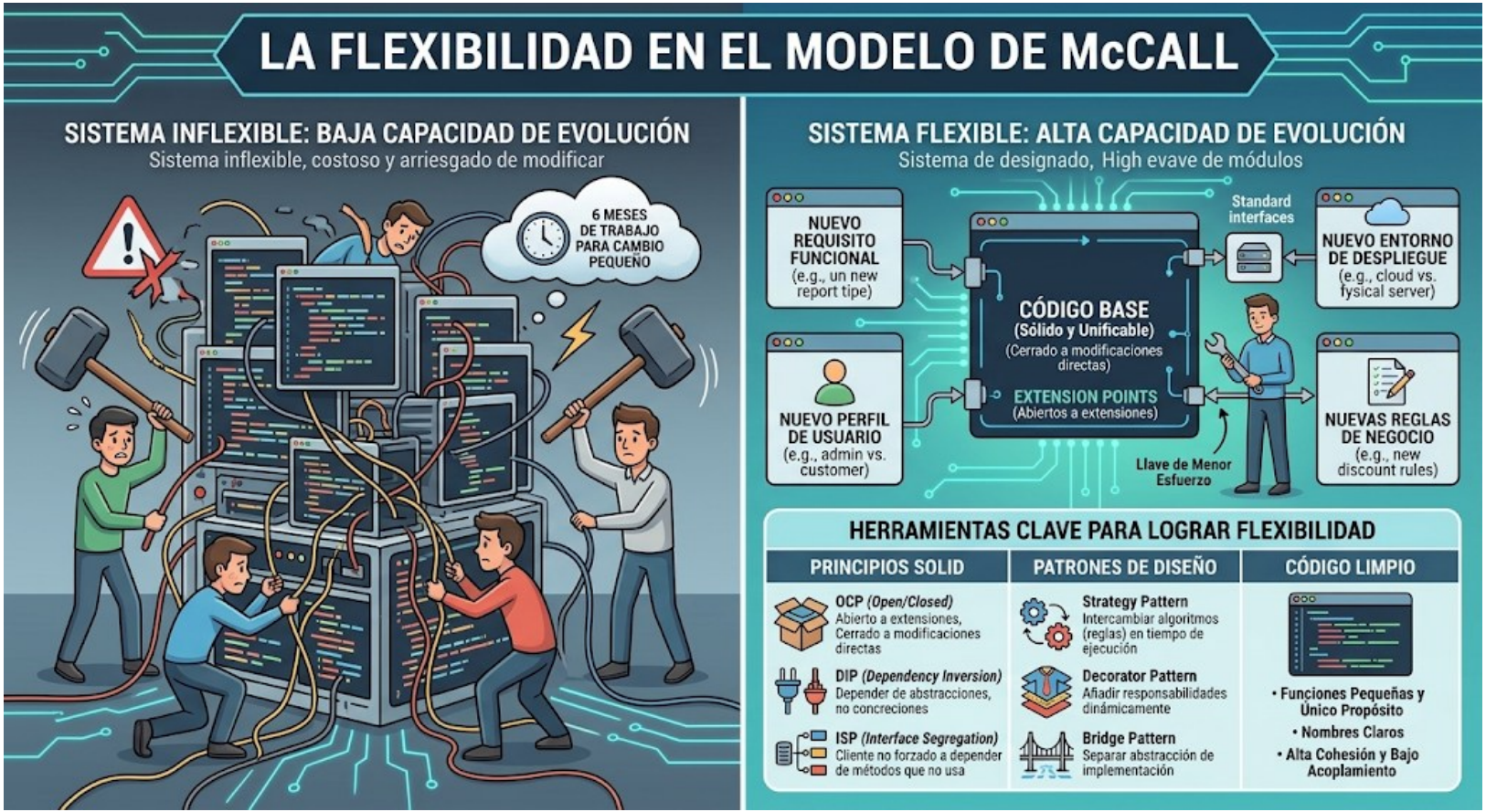
- **Flexibility (Flexibilidad)**: El esfuerzo requerido para modificar un programa operativo con el fin de adaptarlo a **nuevos requisitos funcionales o a cambios en el dominio del negocio** (entorno o contexto de uso). Un sistema altamente flexible aplica **principios de diseño como el Abierto/Cerrado** (OCP), permitiendo **introducir nuevas lógicas sin alterar sustancialmente el código base existente**.

- Ejemplos de cambios frente a los que debe responder un sistema flexible:
  - ✓ nuevos requisitos funcionales;
  - ✓ nuevos entornos de despliegue;
  - ✓ nuevos perfiles de usuario;
  - ✓ nuevas reglas de negocio.
- Un sistema que funciona perfectamente hoy, pero que necesita seis meses de trabajo para incorporar un cambio aparentemente pequeño, presenta una baja capacidad de evolución, independientemente de lo bien que funcione en su estado actual.

1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall

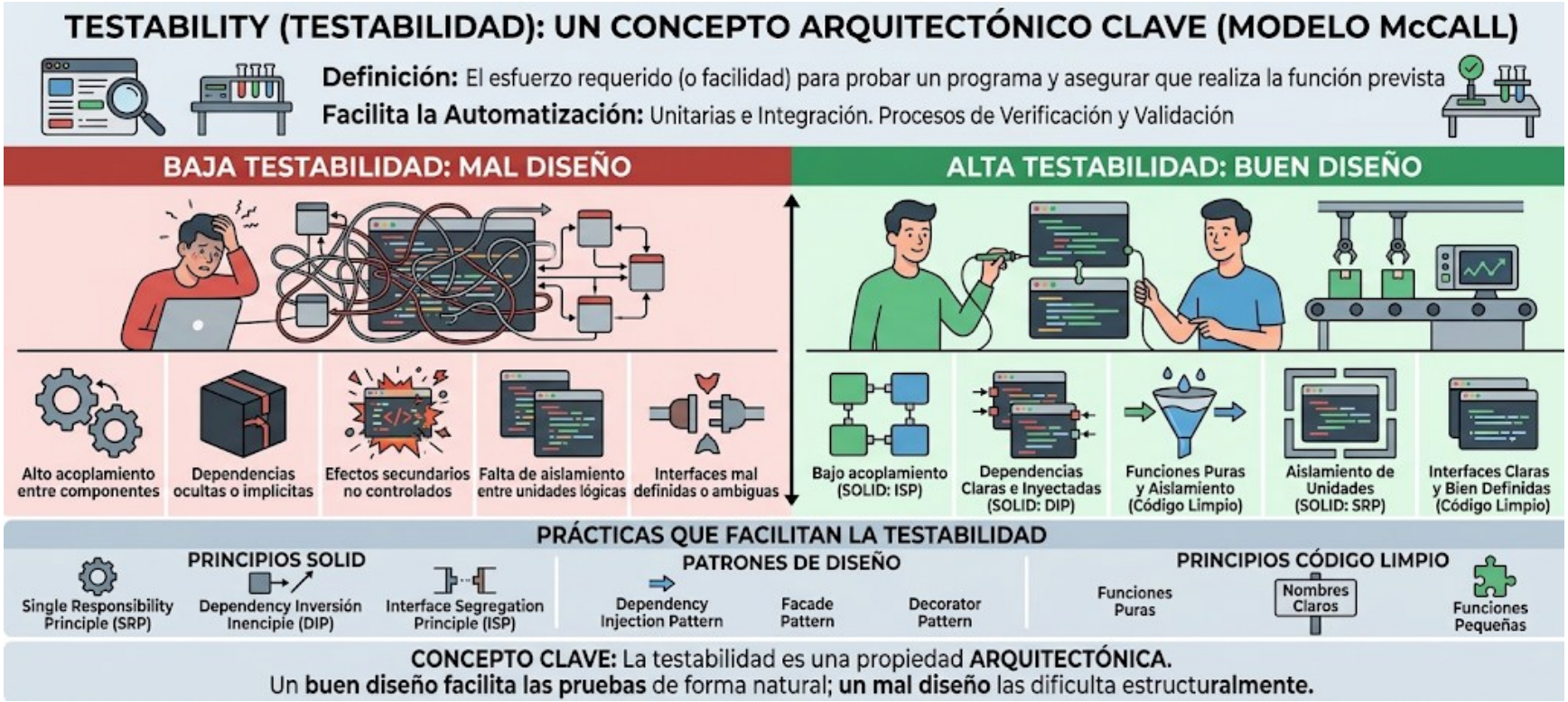
Revisión del producto

- **Testability (Testabilidad)**: El esfuerzo requerido (o la facilidad) para probar un programa (o alguno de sus componentes) y asegurar que realiza la función prevista (procesos de Verificación y Validación). Un software con alta testabilidad facilita la automatización de pruebas unitarias y de integración.

- Un sistema difícil de probar normalmente presenta también problemas estructurales subyacentes, tales como:
  - ✓ alto acoplamiento entre componentes;
  - ✓ dependencias ocultas o implícitas;
  - ✓ efectos secundarios no controlados;
  - ✓ falta de aislamiento entre unidades lógicas;
  - ✓ interfaces mal definidas o ambiguas.

Por esta razón, **la testabilidad no es únicamente una propiedad de "tener pruebas"**, sino también, y de forma muy directa, ***una propiedad arquitectónica***: ***un buen diseño facilita las pruebas de forma natural; un mal diseño las dificulta estructuralmente,*** por muchas herramientas de testing que se utilicen.

1.3 Los 11 factores de Calidad según McCall



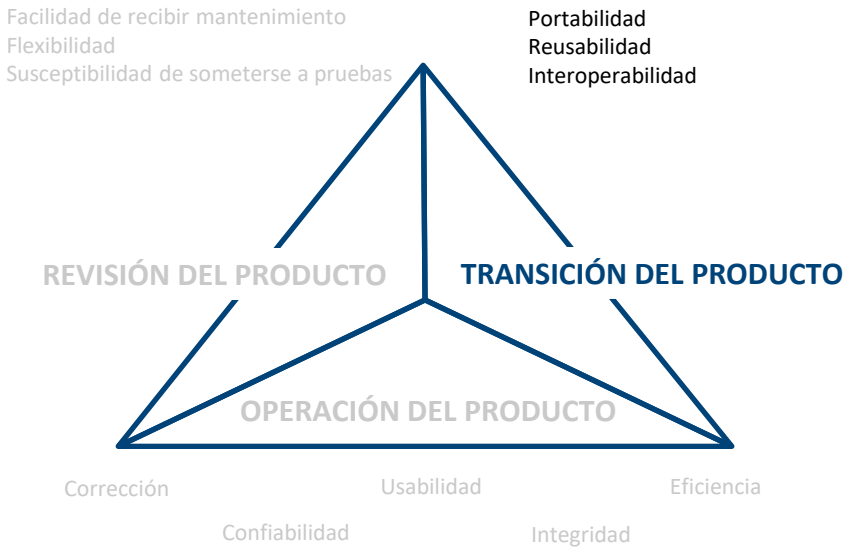
1.3 Los 11 factores de Calidad según McCall

Transición del producto



Figura 15

Diagrama de los factores de la calidad de McCall



Nota. Esta figura muestra todos los factores de calidad según McCall

1.3 Los 11 factores de Calidad según McCall

Transición del producto

Evalúa la capacidad del software para ser integrado, reutilizado o migrado en diferentes infraestructuras. Evalúa el sistema desde una perspectiva arquitectónica a largo plazo, considerando la obsolescencia del hardware subyacente y la necesidad de interoperar en arquitecturas heterogéneas.

- **Portability (Portabilidad)**: El ***esfuerzo requerido (facilidad) para transferir el programa de un entorno de hardware, sistema operativo o entorno de ejecución a otro***. Conceptos asociados: Grado de independencia del sistema operativo/hardware subyacente, adaptabilidad y facilidad de instalación.

En la actualidad puede implicar la capacidad de funcionar correctamente en distintos:

- sistemas operativos; arquitecturas de hardware; proveedores de nube (cloud providers);
- plataformas; navegadores; infraestructuras en general.

Este factor marca ya el inicio de una transición conceptual hacia lo que, como se verá más adelante, ISO/IEC 25010:2023 denomina flexibility (flexibilidad), una categoría más amplia que engloba la portabilidad tradicional junto con otras capacidades de adaptación.

1.3 Los 11 factores de Calidad según McCall



1.3 Los 11 factores de Calidad según McCall

Transición del producto

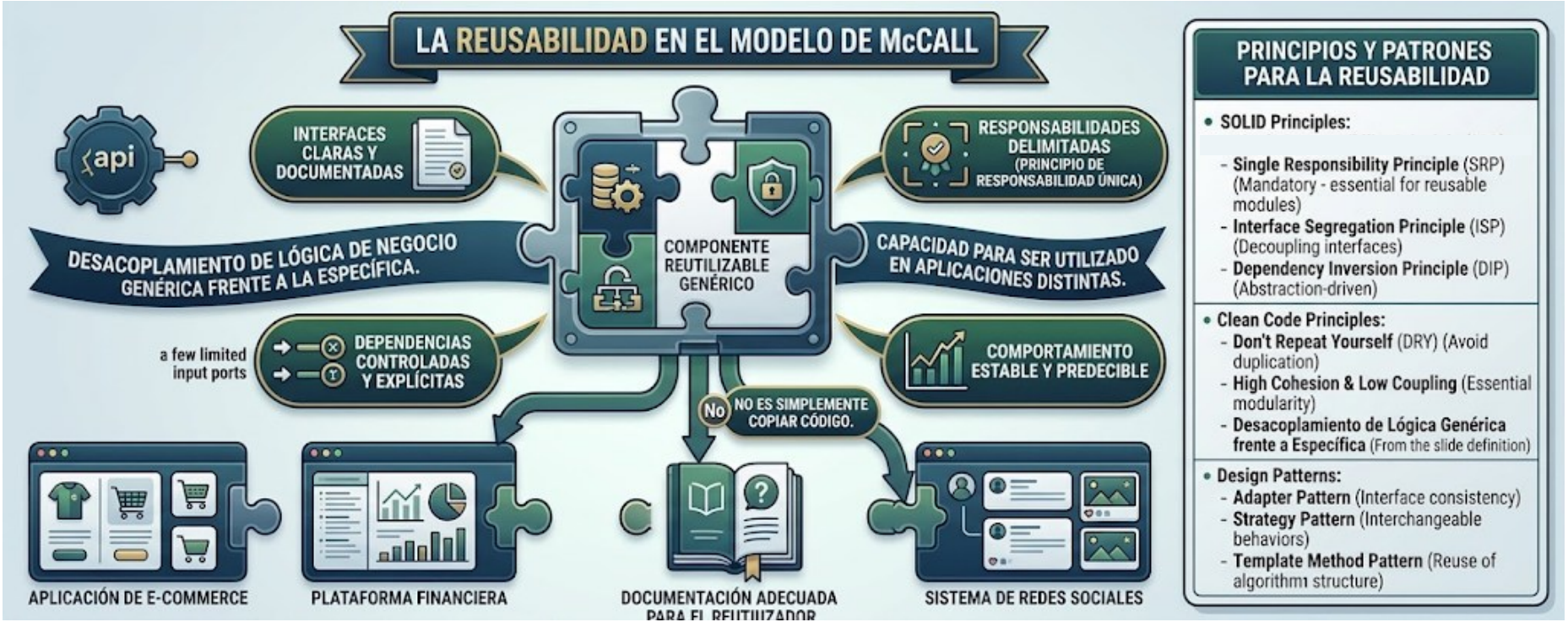
- **Reusability (Reusabilidad)**: La medida o capacidad en que un componente o módulo de software puede ser utilizado en aplicaciones distintas a aquellas para las que fue diseñado originalmente. Implica el desacoplamiento de la lógica de negocio genérica frente a la específica, abstracción de componentes.

Es importante subrayar que la reusabilidad no significa simplemente copiar código de un lugar a otro.

Un componente realmente reutilizable debe presentar:

- ✓ interfaces claras y bien documentadas;
- ✓ responsabilidades bien delimitadas (principio de responsabilidad única);
- ✓ dependencias controladas y explícitas;
- ✓ comportamiento estable y predecible;
- ✓ documentación adecuada para quien vaya a reutilizarlo.

1.3 Los 11 factores de Calidad según McCall



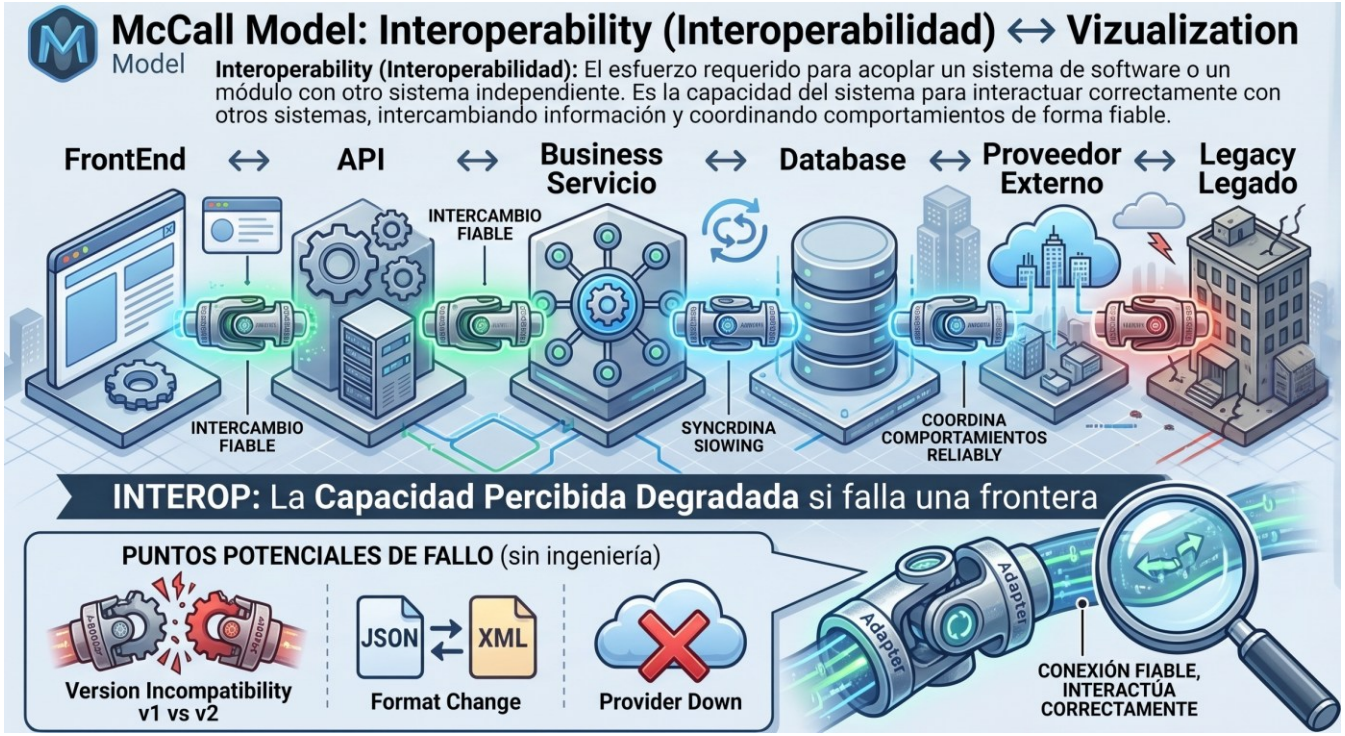
1.3 Los 11 factores de Calidad según McCall

Transición del producto

- **Interoperability (Interoperabilidad)**: El esfuerzo requerido para acoplar un sistema de software o un módulo con otro sistema independiente. Es la capacidad del sistema para interactuar correctamente con otros sistemas, intercambiando información y coordinando comportamientos de forma fiable.

- En las arquitecturas modernas esta propiedad resulta crítica, ya que prácticamente ninguna aplicación empresarial existe de manera aislada. Un ejemplo típico de cadena de dependencias:
- FrontEnd ↔ API ↔ Servicio Negocio ↔ BB DD ↔ Proveedor Externo ↔ Sistema Legado
- Cada una de estas fronteras (representadas por los símbolos “↔ ") constituye un punto potencial de fallo de calidad: un cambio de formato, una incompatibilidad de versiones o una caída del proveedor externo puede degradar la calidad percibida del sistema completo, aunque cada componente individual funcione correctamente por separado.

1.3 Los 11 factores de Calidad según McCall



PRÁCTICAS E INGENIERÍA PARA LOGRAR INTEROPERABILIDAD

PRINCIPIOS DE DISEÑO SOLID NECESARIOS

SRP	MANTENER RESPONSABILIDADES ÚNICAS Y ENFOCADAS EN CADA CLASE, EVITANDO MÚLTIPLES TAREAS.
ISP	EVITAR INTERFACES SOBRECARGADAS; CREAR INTERFACES PEQUEÑAS Y ESPECÍFICAS PARA LO QUE SE NECESITA.
DIP	DEPENDER DE ABSTRACCIONES (INTERFACES), NO DE IMPLEMENTACIONES CONCRETAS.

PATRONES DE DISEÑO CLAVE

Adapter	ADAPTAR INTERFACES INCOMPATIBLES PARA QUE PUEDAN TRABAJAR JUNTAS.
Facade	PROPORCIONAR UNA INTERFAZ UNIFICADA Y SIMPLIFICADA PARA SUBSISTEMAS COMPLEJOS.
Proxy	UN OBJETO SUSTITUTO QUE CONTROLA Y VALIDA EL ACCESO A OTRO OBJETO.
Gateway	ESTANDARIZAR Y CENTRALIZAR EL ACCESO A SERVICIOS EXTERNOS DE RED.

PRINCIPIOS DE CÓDIGO LIMPIO Y OTRAS PRÁCTICAS

Bajo Acoplamiento (Inter-sistema)	INTERFACES CLEAN, ACOPABILA, DEPENDENCIAS EXTORNOS.
Versioning de APIs	MANEJAR CAMBIOS DE FORMA CONTROLADA MEDIANTE NÚMEROS DE VERSIÓN.
Resilience Patterns	IMPLEMENTAR MECANISMOS PARA QUE EL SISTEMA SE RECUPERE DE FALLOS DE RED Y PROVEEDORES EXTERNOS.
Api Contract First Design	DEFINIR Y DOCUMENTAR LA API ANTES DE SU IMPLEMENTACIÓN, ASEGURANDO LA ALINEACIÓN.
Schema Validation	ASEGURAR QUE LOS DATOS INTERCAMBIADOS CUMPLAN CON UNA ESTRUCTURA DEFINIDA.